Cloud Native Application Delivery

CICD with GitOps

Copy page

This article explains GitOps methodology for managing Kubernetes infrastructure and applications using Git as the single source of truth for deployments.

GitOps has become a leading methodology for managing Kubernetes infrastructure and applications by utilizing Git as the single source of truth. This approach uses declarative configuration and automated delivery to enable rapid, predictable, and safe deployments.

In a typical GitOps workflow, two Git repositories are employed:

> One repository holds the application code, resource definitions, configuration data, and container images.
>
> The other repository contains the Kubernetes manifests. These YAML files detail the desired state of the cluster, including key resources such as Deployments, Services, Ingresses, ConfigMaps, and Secrets.

Once the repositories are set up, an operator like **ArgoCD** is deployed inside the Kubernetes cluster. ArgoCD continuously monitors the manifests repository for changes and reconciles the actual state of the cluster with the desired state by automatically deploying updates and managing resource lifecycle events.

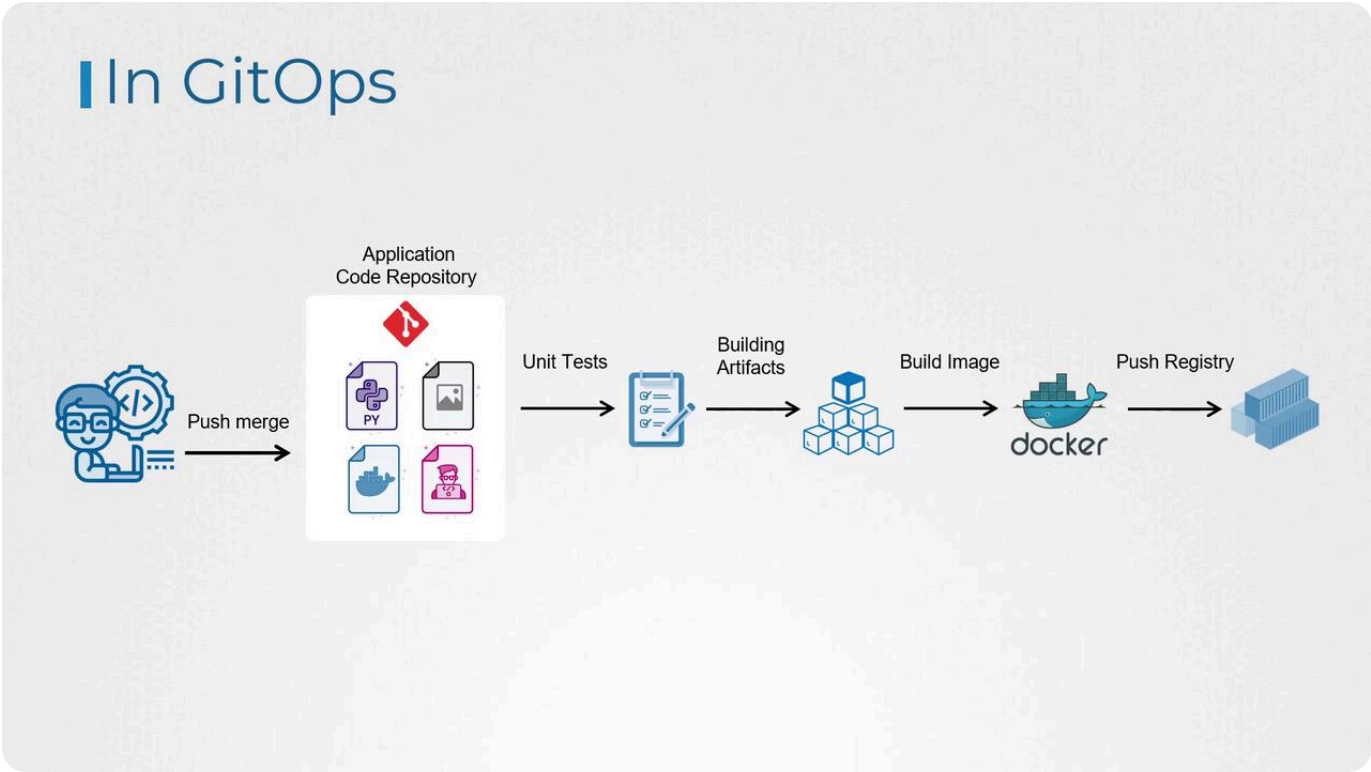
Consider this common scenario:

1. A developer commits new changes to the application code repository.
2. A CI pipeline is triggered, executing several steps:

    Running unit tests

    Building artifacts

KodeKloud



After the new image is available in the container registry, the Kubernetes manifests repository is updated with the new image version. The YAML files referencing the image are modified and the changes are committed to the repository. A pull request is then created to merge these updates into the repository's master branch. After thorough review and approval by a project manager or architect, the pull request is merged.

Once merged, ArgoCD automatically detects these changes and syncs the cluster's state to match the desired configuration by deploying the updated application version. This process ensures a safe and controlled deployment across your cluster.

> Beyond automated deployments, GitOps with ArgoCD provides a reliable rollback mechanism. In case of issues or failures, you can revert to a previously stable version using the following commands:

KodeKloud

In this lesson, we will explore how to implement this pipeline using Git, ArgoCD, and Kubernetes. Although our primary focus is on continuous deployment (CD) with ArgoCD, the overall process is integrated with a CI pipeline that manages testing and image building.

For more details on implementing GitOps practices, consider reviewing **Kubernetes Basics** and other relevant documentation.

Watch Video

Learn more

Was this page helpful? Yes No

Push vs Pull based Deployments
Previous

Git Repositories Dockerfile and Application Walkthrough
Next

KodeKloud